# STM32F103C8T6 定时器与 PWM 实验报告

STM32F103C8T6 最小系统板 · 嵌入式系统实验

| 项目 | 内容 |
| --- | --- |
| **实验名称** | STM32 定时器原理与 PWM 呼吸灯设计 |
| **姓　　名** | 徐正昕 |
| **学　　号** | 632407030612 |
| **班　　级** | 通信 2403 |
| **实验日期** | 2026 年 10 月 19 日 |
| **实验平台** | STM32F103C8T6 最小系统板 + Keil MDK 5.36 |
| **代码仓库** | https://github.com/xude111/xude11 |

## 一、实验目的

1. 深入了解 STM32 通用定时器（TIM2~TIM5）的内部结构与工作原理；
2. 掌握时基单元 **PSC（预分频）、CNT（计数器）、ARR（自动重装载）** 的配合关系，会用它们算出需要的定时时间；
3. 掌握定时器**更新中断**的编程方法，用中断实现精确定时（2 s 周期的 LED 亮灭）；
4. 掌握定时器 **PWM 输出模式**的原理与配置方法（通道、极性、占空比）；
5. 用 PWM + 占空比扫描实现**呼吸灯**，并用 Keil 软件仿真的**逻辑分析仪（虚拟示波器）**观察输出波形，验证频率与占空比。

## 二、实验器材

| 序号 | 名称 | 规格 / 说明 |
| --- | --- | --- |
| 1 | STM32 最小系统板 | STM32F103C8T6（72 MHz） |
| 2 | LED + 限流电阻 | 发光二极管 1 只、220 Ω 1 只 |
| 3 | 下载器 | ST-Link V2（SWD） |
| 4 | 杜邦线 | 若干 |
| 5 | 软件 | Keil MDK 5.36 + STM32 HAL 库 |

## 三、实验原理

### 3.1 通用定时器的基本结构

STM32F103 的 TIM2~TIM5 是 16 位通用定时器，核心是一个**时基单元**：

```
TIMxCLK ──▶ [PSC 预分频器] ──▶ CK_CNT ──▶ [CNT 计数器] ──▶ 溢出/比较 ──▶ 中断 / PWM
                ÷(PSC+1)                    计数到 ARR 归零
```

- **PSC（预分频）**：把定时器时钟再分频，得到计数频率 f_CNT = f_TIM / (PSC+1)；
- **CNT**：在 0 ~ ARR 之间循环计数；
- **ARR（自动重装载）**：计数器数到 ARR 后回到 0，并产生**更新事件**（可触发中断）；
- 定时（溢出）时间：T = (PSC+1) × (ARR+1) / f_TIM。

**关键点**：TIM3 挂在 APB1 上，APB1 分频为 2 时 PCLK1 = 36 MHz，但**定时器时钟 = 2 × PCLK1 = 72 MHz**，所以计算时要用 72 MHz。

### 3.2 PWM 输出模式

PWM 模式 1：计数器 CNT 与比较寄存器 **CCR** 比较 ——

```
CNT < CCR  → 通道输出有效电平（高）
CNT ≥ CCR  → 输出无效电平（低）
```

于是：

```
占空比 D = CCR / (ARR + 1)
PWM 频率 f = f_TIM / ((PSC+1) × (ARR+1))
```

**呼吸灯**就是让 CCR 从 0 缓慢增加到 ARR，再缓慢减回 0：占空比越大 LED 越亮，形成"渐亮—渐灭"的效果。

---

## 四、硬件连接

| 模式 | 连接方式 |
| --- | --- |
| 任务 1（定时中断） | 直接用**板载 LED**：PC13（低电平点亮），无需接线 |
| 任务 2（PWM） | **PA6（TIM3_CH1）** → 220 Ω → LED 正极，LED 负极 → GND |

> PA6 是 TIM3 通道 1 的默认复用引脚；若使用重映射引脚需额外配置 AFIO。

---

## 五、任务 1：定时器计数方式，LED 2 s 周期亮灭

### 5.1 参数计算

要求 LED 亮 1 s、灭 1 s，即 **2 s 一个周期**，所以定时器每 **1 s** 产生一次更新中断，在中断里翻转 LED。

取计数频率 10 kHz（便于计算）：

```
PSC = 72 000 000 / 10 000 − 1 = 7200 − 1 = 7199
ARR = 10 000 − 1 = 9999          → 10 kHz / 10000 = 1 Hz
T   = 1 s                        → 翻转一次
周期 = 2 × 1 s = 2 s ✓
```

### 5.2 关键代码

```
htim3.Instance = TIM3;
htim3.Init.Prescaler = 7200 - 1;      /* 10 kHz */
htim3.Init.Period    = 10000 - 1;     /* 1 s 溢出 */
htim3.Init.CounterMode = TIM_COUNTERMODE_UP;
HAL_TIM_Base_Init(&htim3);

/* MSP 中：__HAL_RCC_TIM3_CLK_ENABLE(); 并使能 TIM3_IRQn */
HAL_TIM_Base_Start_IT(&htim3);        /* 启动更新中断 */

/* 中断回调：每 1 s 进来一次 */
void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim)
{
    if (htim->Instance == TIM3)
        HAL_GPIO_TogglePin(GPIOC, GPIO_PIN_13);   /* 翻转 LED */
}
```

### 5.3 现象

烧录后板载 LED **亮 1 秒、灭 1 秒**，用秒表计时 10 个周期约 20 s，周期误差在肉眼/秒表精度内看不出来，说明定时精度良好（误差来自 HSE 晶振本身的精度，典型 ±20 ppm）。

---

## 六、任务 2：PWM 呼吸灯

### 6.1 参数计算

取 PWM 频率 1 kHz（远高于人眼闪烁频率，LED 不会闪）：

```
PSC = 72 000 000 / 1 000 000 − 1 = 71
ARR = 1 000 − 1 = 999          → f_PWM = 1 MHz / 1000 = 1 kHz
占空比分辨率 = 1/1000 = 0.1%
```

**呼吸周期**：CCR 每 1 ms 变化 1 个步进，0→999 用 1 s，999→0 再用 1 s，合计 **2 s** ✓

### 6.2 关键代码

```
htim3.Init.Prescaler = 72 - 1;        /* 1 MHz */
htim3.Init.Period    = 1000 - 1;      /* 1 kHz PWM */
HAL_TIM_Base_Init(&htim3);
HAL_TIM_PWM_Init(&htim3);

TIM_OC_InitTypeDef sConfigOC = {0};
sConfigOC.OCMode     = TIM_OCMODE_PWM1;   /* CNT < CCR 输出高 */
sConfigOC.Pulse      = 0;                 /* 初始占空比 0 */
sConfigOC.OCPolarity = TIM_OCPOLARITY_HIGH;
HAL_TIM_PWM_ConfigChannel(&htim3, &sConfigOC, TIM_CHANNEL_1);

/* MSP 中把 PA6 配成 GPIO_MODE_AF_PP */
HAL_TIM_PWM_Start(&htim3, TIM_CHANNEL_1);

/* 主循环：呼吸 */
while (1) {
    for (uint16_t i = 0; i <= 999; i++) {          /* 渐亮 1 s */
        __HAL_TIM_SET_COMPARE(&htim3, TIM_CHANNEL_1, i);
        HAL_Delay(1);
    }
    for (uint16_t i = 999; i > 0; i--) {           /* 渐灭 1 s */
        __HAL_TIM_SET_COMPARE(&htim3, TIM_CHANNEL_1, i);
        HAL_Delay(1);
    }
}
```

### 6.3 现象

LED 由暗逐渐变亮、再逐渐变暗，**一个完整呼吸周期约 2 s**，看不到闪烁（因为 PWM 频率 1 kHz 远高于人眼能分辨的闪烁频率）。变化过程平滑，占空比 0.1% 的分辨率足够。

---

## 七、Keil 虚拟示波器观察 PWM 波形

### 7.1 操作步骤

```
① 工程里把 TIM_MODE 设为 2U（PWM 模式），重新编译
② 魔术棒 → Debug → 左侧选 "Use Simulator"
③ Ctrl+F5 进入仿真
④ View → Analysis Windows → Logic Analyzer
⑤ 点 Setup... → New，信号名填  PORTA.6   → Display Type 选 Bit
```

⑥ F5 运行，即可看到 PA6 上的 PWM 方波

> 信号名要写 **PORTA.6**（Keil 逻辑分析仪的命名规则），写成 GPIOA.ODR.6 会提示 Unknown Signal。

### 7.2 波形分析

- **周期**：两个上升沿之间约 **1 ms**（对应 1 kHz），与 f = 72 MHz / (72 × 1000) 一致；
- **占空比**：高电平宽度随时间缓慢变化 —— 从几乎全低（LED 暗）逐渐变宽到几乎全高（LED 最亮），再逐渐变窄；
- **呼吸周期**：观察占空比从最小到最大再回最小，约 **2 s**，与程序中 1000 步 × 1 ms × 2 相符；
- **有无毛刺**：波形边沿干净，说明 ARR 预装载（TIM_AUTORELOAD_PRELOAD_ENABLE）有效，运行中改 CCR 不会产生异常脉冲。

---

## 八、遇到的问题与解决方法

| 序号 | 问题 | 原因 | 解决 |
|---|---|---|---|
| 1 | 定时时间差了 2 倍 | 误把 TIM3 时钟当成 36 MHz | APB1 分频≠1 时定时器时钟 = 2×PCLK1 = **72 MHz** |
| 2 | LED 不亮 | 板载 PC13 是**共阳、低电平点亮** | 初始化时输出高电平熄灭，翻转逻辑不受影响 |
| 3 | PWM 引脚无输出 | 忘记把 PA6 配成复用推挽 | MSP 中 GPIO_MODE_AF_PP + __HAL_RCC_TIM3_CLK_ENABLE() |
| 4 | 呼吸灯有轻微闪烁 | PWM 频率太低（如 100 Hz） | 提高到 1 kHz 以上 |
| 5 | 呼吸过程有跳变、不平滑 | 每步延时太短或步进太大 | 每 1 ms 改 1 个步进（1000 级），并在低亮度区适当加密 |
| 6 | 仿真时逻辑分析仪没信号 | 信号名写错 | 用 PORTA.6 |

---

## 九、实验总结

1. **定时器的本质是"分频 + 计数 + 比较"**：PSC 决定计数快慢，ARR 决定计到多少归零。只要记住 T = (PSC+1)(ARR+1)/f_TIM，任何定时时间都能反算出来。
2. **中断方式 vs 循环延时**：用 HAL_Delay() 也能闪灯，但 CPU 被占住且时间不准；定时器中断由硬件计时，CPU 只在时间到时进来一下，精度和效率都高得多。
3. **PWM 是"用数字方式模拟模拟量"**：改变 CCR 就改变了高电平的比例，LED 的亮度、电机的转速、舵机的角度都由它控制 —— 这是嵌入式里最常用的"软模拟输出"手段。
4. **呼吸灯的关键是变化速度**：PWM 频率要足够高（> 200 Hz）才不闪，占空比的变化要足够慢（1~2 s 一个来回）人眼才觉得"呼吸"，两者缺一不可。
5. **没有示波器也能看波形**：Keil 的软件仿真 + 逻辑分析仪能精确给出周期和占空比，本次测得 PWM 周期 ≈ 1 ms、呼吸周期 ≈ 2 s，与理论计算完全吻合。

## 附录：工程文件与编译结果

| 文件 | 说明 |
| --- | --- |
| Core/Src/main.c | 两种模式的实现，用 TIM_MODE 宏切换 |
| Core/Src/stm32f1xx_hal_msp.c | TIM3 时钟、PA6 复用推挽、TIM3 中断 |
| Core/Src/stm32f1xx_it.c | TIM3_IRQHandler、SysTick_Handler |
| MDK-ARM/TIM_PWM.uvprojx | Keil 工程 |
| MDK-ARM/TIM_PWM_mode1.hex | 任务 1：定时中断闪灯 |
| MDK-ARM/TIM_PWM_mode2.hex | 任务 2：PWM 呼吸灯 |

```
模式 1 定时中断：  Code=3744  RO=304  RW=16  ZI=1096   0 Error 0 Warning
模式 2 PWM 呼吸：  Code=4708  RO=304  RW=16  ZI=1096   0 Error 0 Warning
```